Algoritmos de ordenação

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Algoritmos e Estruturas de Dados II
Prof. Fabrício Sérgio de Paula

Tópicos

- Introdução
- Ordenação por bolha (bubble sort)
- Ordenação por inserção (insertion sort)
- Ordenação por intercalação (mergesort)
- Ordenação rápida (quicksort)
- Ordenação em heap (heapsort)
- Limite inferior para algoritmos de ordenação
- Exercícios

Introdução

- Ordenação: problema que aparece como pré-processamento de um grande número de aplicações
  - Ex.: Agilizar busca, listar candidatos aprovados em concurso por nome (ou número de inscrição/pontuação), apresentar cheques processados por número (ou data de compensação), etc.
- Problema da ordenação
  - Entrada: seqüência de n números $<a_1, a_2, ..., a_n>$
  - Saída: permutação $<a_1', a_2', ..., a_n'>$, onde $a_1' \leq a_2' \leq ... \leq a_n'$

Introdução

- Conceitos:
  - Ordenação no local (in place): quantidade de memória temporária exigida é constante
  - Ordenação estável: ao término, elementos iguais estão na mesma ordem em que estavam na entrada
- Existem diversos algoritmos: alguns melhores que outros em determinadas situações
  - Fatores a considerar: complexidade de tempo, quantidade de dados, quantidade de memória exigida, simplicidade de implementação

Ordenação por bolha (bubble sort)

- Método bastante simples e difundido
- Funcionamento:
  - Percorrer vetor do início ao fim diversas vezes
    - Comparar elementos consecutivos L[j] e L[j+1]
      - Trocá-los sempre que estiverem fora de ordem, isto é, L[j] > L[j+1
- Intuitivamente, a cada percurso do início ao fim do vetor, o maior elemento ainda não ordenado vai para o lugar correto
  - São necessárias até n-1 iterações

Ordenação por bolha (bubble sort)

- Algoritmo:

```
para i = 1, ... n faça
    para j = 1, ... n − 1 faça
        se L[j] . chave > L[j + 1] . chave então
            trocar(L[j], L[j + 1])
```

- Complexidade: $O(n^2)$ em qualquer caso
  - Pode fazer muitas comparações desnecessárias (ex.: vetor já ordenado)

Ordenação por bolha (bubble sort)

- Duas melhorias podem ser feitas:
  - Elementos já posicionado no final do vetor (maiores) não precisam ser comparados: estão na posição correta
  - Se não houver nenhuma troca durante uma iteração i, o vetor já está ordenado
    - Algoritmo é encerrado

Ordenação por bolha (bubble sort)

- Algoritmo melhorado:

```
mudou := V; n' := n; guarda := n
enquanto mudou faça
    j := 1; mudou := F
    enquanto j < n' faça
        se L[j] . chave > L[j + 1] . chave então
            trocar(L[j], L[j + 1])
            mudou := V
            guarda := j
        j := j + 1
    n' := guarda
```

Ordenação por bolha (bubble sort)

- Complexidade do algoritmo melhorado:
  - Pior caso: $O(n^2)$
  - Melhor caso: $O(n)$
- Exemplo de execução:

Iteração	Tabela	Trocas
tabela inicial	40 37 95 42 39 51 60	
após 1ª iteração	37 40 42 39 51 60 95	5
após 2ª iteração	37 40 39 42 51 60 95	1
após 3ª iteração	37 39 40 42 51 60 95	1
após 4ª iteração	37 39 40 42 51 60 95	0

Ordenação por inserção (insertion sort)

- Método semelhante à inserção de uma nova carta em um conjunto de cartas já ordenadas
- Funcionamento:
  - L[1..1] é um subvetor já ordenado
  - Insira L[2] na posição correta em L[1..1]
  - Insira L[3] na posição correta em L[1..2]
  - Insira L[4] na posição correta em L[1..3]
  - ...
  - Insira L[n] na posição correta em L[1..n-1]

Ordenação por inserção (insertion sort)

- Algoritmo:

```
para i = 2, ... n faça
        prov := L[i]; valor := L[i] . chave
        j := i − 1
        enquanto j ≥1 e valor < L[j] . chave faça
                L[j + 1] := L[j]
                j := j − 1
        L[j + 1] := prov
```

- Complexidade varia de $O(n)$ a $O(n^2)$

Ordenação por inserção (insertion sort)

- Exemplo de execução:

Iteração	Tabela	Trocas
tabela inicial	40 37 95 42 23 51 27	
após $i = 2$	37 40 95 42 23 51 27	1
após $i = 3$	37 40 95 42 23 51 27	0
após $i = 4$	37 40 42 95 23 51 27	1
após $i = 5$	23 37 40 42 95 51 27	4
após $i = 6$	23 37 40 42 51 95 27	1
após $i = 7$	23 27 37 40 42 51 95	5

Ordenação por intercalação (mergesort)

- Método que consiste em intercalar duas listas ordenadas
- Funcionamento:
  - Algoritmo divide, recursivamente, lista de entrada de tamanho $n > 1$ em duas de tamanho $\approx n/2$
  - Na volta da recursão, listas ordenadas são intercaladas, duas a duas:
    - A partir das listas de tamanho 1 (portanto, ordenadas) são construídas listas de tamanho 2 ordenadas
    - Listas ordenadas maiores são construídas através da intercalação de listas menores já ordenadas

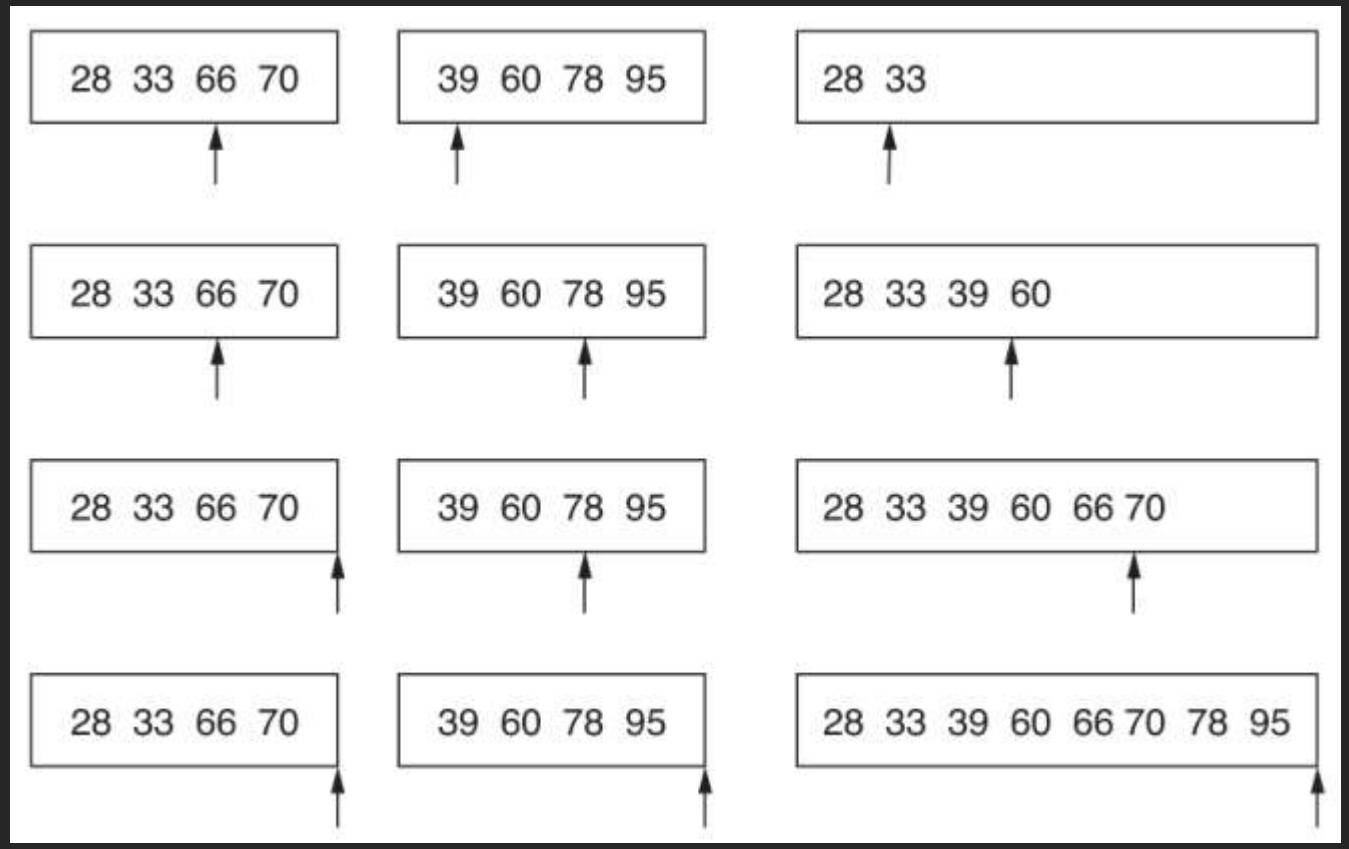
Ordenação por intercalação (mergesort)

- Procedimento intercalar:
  - Entrada: duas listas no vetor L
    - L[ini1..ini2-1] e L[ini2..fim2]
  - Saída em L[ini1..fim2]
    - Uso de vetor temporário Tmp
- Complexidade de tempo: O(n), onde n é a soma do tamanho das listas de entrada

```
procedimento intercalar(L, Tmp, ini1, ini2, fim2);
    fim1 := ini2 − 1; nro := 0
    ind := ini1
        enquanto (ini1 ≤ fim1) e (ini2 ≤ fim2) faça
            se L[ini1] . chave < L[ini2] . chave então
                Tmp[ind] := L[ini1]
                ini1 := ini1 + 1
            senão Tmp[ind] := L[ini2]
                ini2 := ini2 + 1
            ind := ind + 1; nro := nro + 1
        enquanto {ini1 ≤ fim1} faça
            Tmp[ind] := L[ini1]
            ini1 := ini1 + 1; ind := ind + 1; nro := nro + 1
        enquanto {ini2 ≤ fim2} faça
            Tmp[ind] := L[ini2]
            ini2 := ini2 + 1; ind := ind + 1; nro := nro + 1
        para i := 1, ..., nro faça
            L[i + ~~ini1~~ − 1] := Tmp[i + ~~ini1~~ − 1]
```

Ordenação por intercalação (mergesort)

- Exemplo de intercalação:



Ordenação por intercalação (mergesort)

- Algoritmo mergesort:

```
procedimento mergesort(esq, dir);
    se esq < dir então
        centro := ⌊(esq + dir)/2⌋;
        mergesort(esq, centro);
        mergesort(centro + 1, dir);
        intercalar(esq, centro + 1, dir);
```

- Chamada externa: mergesort(1, n)

Ordenação por intercalação (mergesort)

- Exemplo de execução (volta da recursão):

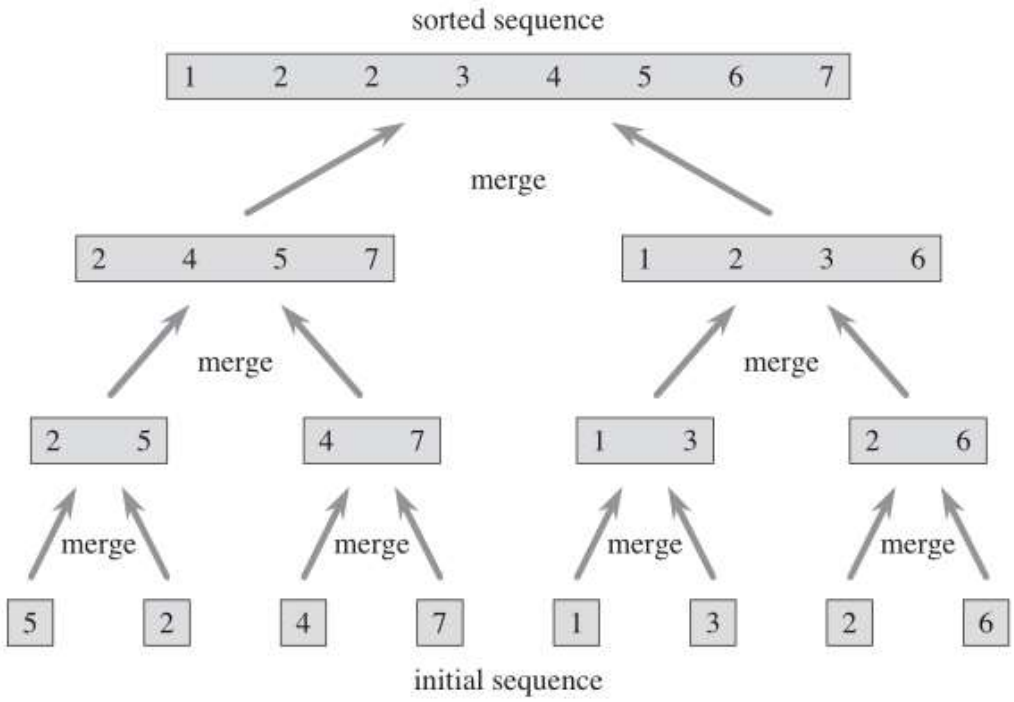
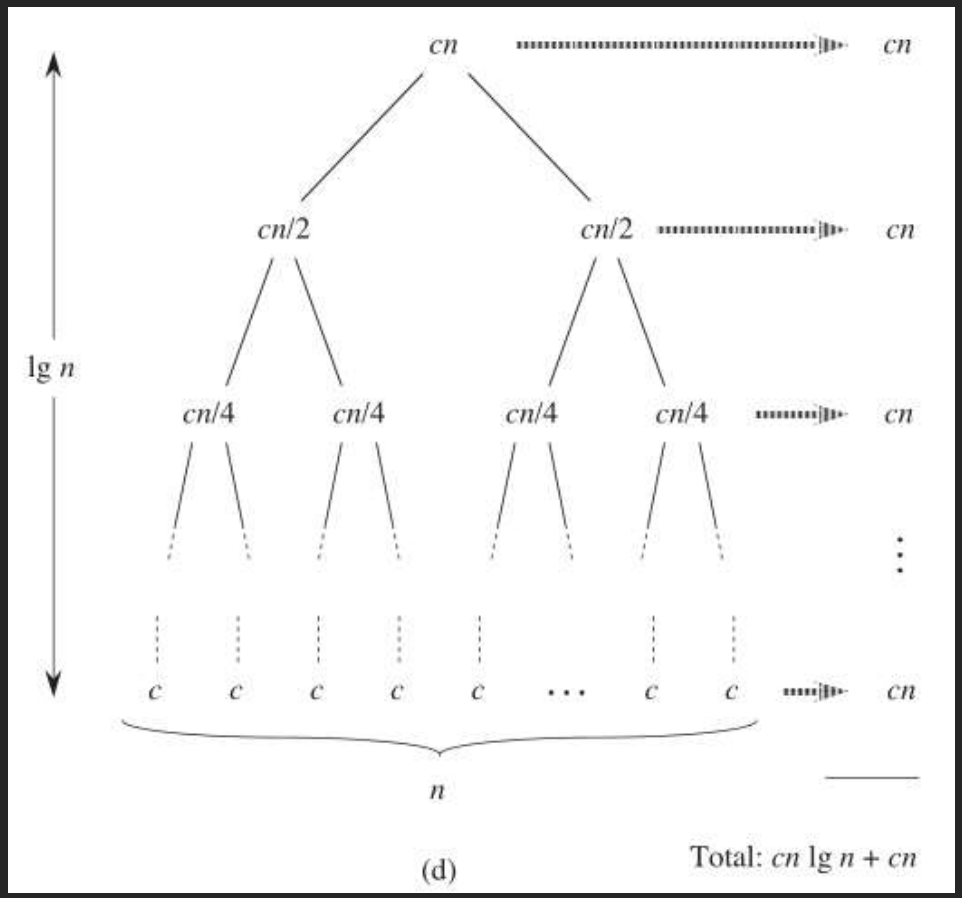


Figure 2.4 The operation of merge sort on the array $A = \langle 5, 2, 4, 7, 1, 3, 2, 6 \rangle$. The lengths of the sorted sequences being merged increase as the algorithm progresses from bottom to top.

Ordenação por intercalação (mergesort)

- Complexidade de tempo: $\Theta(n \lg n)$ em qualquer caso



Ordenação por intercalação (mergesort)

- Considerações sobre Mergesort:
  - Um dos melhores algoritmos do ponto de vista téorico
  - Não faz ordenação no local: gasta $\Theta(n)$ em memória temporária
    - Prejudica uso na prática

Ordenação rápida (quicksort)

- Quicksort: um dos métodos mais eficientes na prática
  - Teoricamente: pior que mergesort e heapsort
- Ordenação recursiva de lista L com n elementos:
  - Se n = 0 ou n = 1, então a lista está ordenada
  - Escolha qualquer elemento x de L: pivô
  - Separe L – {x} em dois conjuntos disjuntos S_1 e S_2:
    - S_1: elementos de L menores que x
    - S_2: elementos de L maiores ou iguais a x
  - Ordene recursivamente S_1 e S_2
  - Lista ordenada: S_1, x, S_2

Ordenação rápida (quicksort)

- Pivô escolhido influencia muito desempenho
  - Pivô ideal (mediana) deixa S_1 e S_2 com tamanhos similares
  - Pivô ruim (menor ou menor elemento): deixa um dos conjuntos vazio e o outro com n-1 elementos
- Exemplos de escolha de pivô:
  - Primeiro/último elemento de L: boa para L aleatória
    - Se estiver ordenada/inversamente ordenada: pivô ruim
  - Escolha aleatória de pivô: “pesa” no tempo de execução
  - Escolha da mediana de L: melhor possível, mas custosa
  - Mediana de três (primeiro, último, central): bons resultados práticos mas pode render particionamento ruim

```
procedimento quicksort(ini, fim)
    se fim − ini < 2 então
        se fim − ini = 1 então
            se L[ini] . chave > L[fim] . chave então
                trocar(L[ini], L[fim]);
    senão PIVO(ini, fim, mediana)
        trocar((L[mediana], L[fim])
        i := ini; j := fim − 1
        key := L[fim] . chave
        enquanto j ≥ i faça
            enquanto L[i] . chave < key faça
                i := i + 1
            enquanto L[j] . chave > key faça
                j := j − 1
            se j ≥ i então
                trocar((L[i], L[j])
                i := i + 1; j := j − 1
        trocar((L[i], L[fim])
        quicksort(ini, i − 1)
        quicksort(i + 1, fim);
```

Ordenação rápida (quicksort)

- Exemplo de particionamento:

40	37	95	42	23	51	27
27	37	95	42	23	51	40
↑					↑	
i					*j*	
27	37	95	42	23	51	40
		↑		↑		
		i		*j*		
27	37	23	42	95	51	40
		↑		↑		
		i		*j*		
27	37	23	42	95	51	40
		↑	↑			
		j	*i*			
[27	37	23]	40	[95	51	42]

Ordenação rápida (quicksort)

- Complexidade do algoritmo:
  - Pior caso: $\Theta(n^2)$
  - Melhor caso: $\Theta(n \log n)$
  - Caso médio: $\Theta(n \log n)$
- Quicksort é in place

Ordenação em heap (heapsort)

- Heapsort realiza ordenação usando heap de máximo
- Algoritmo:

algoritmo 6.7: Ordenação de $\mathcal{T}$

$arranjar(n)$

$m := n$

enquanto $m > 1$ **faça**

$\quad \mathcal{T}[1] \Leftrightarrow \mathcal{T}[m]$

$\quad m := m - 1$

$\quad descer(1, m)$

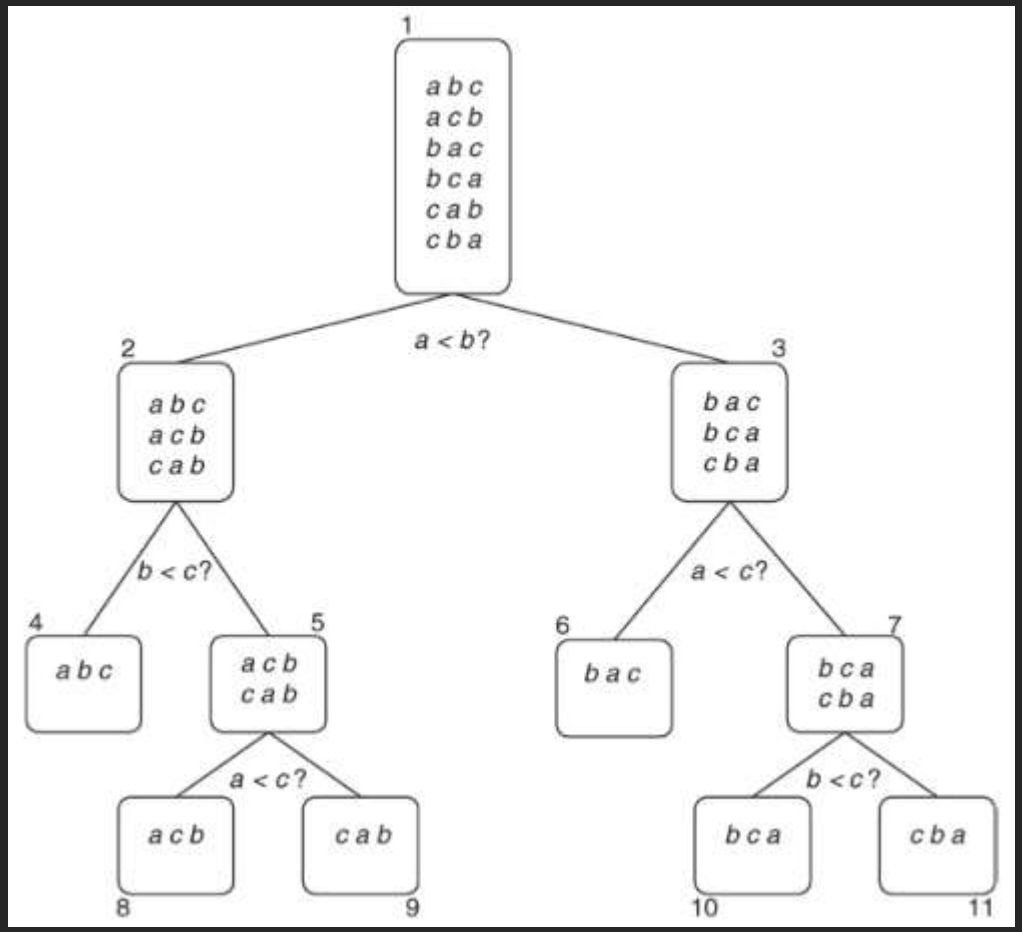
- Complexidade: O(n lg n)

Limite inferior para algoritmos de ordenação

- Limite inferior para algoritmos de ordenação por comparação pode ser obtido usando uma árvore de decisão
- Árvore de decisão: árvore binária onde cada nó representa uma comparação entre dois elementos
  - Varia de acordo com algoritmo e entrada
  - Uma ordenação é obtida através de um caminho da raiz até uma folha
    - Maior caminho (mais comparações): pior caso
    - Menor caminho (menos comparações): melhor caso

Limite inferior para algoritmos de ordenação

- Árvore de decisão para bubble sort com três elementos:



Limite inferior para algoritmos de ordenação

- Seja algoritmo de ordenação por comparação com entrada de tamanho n
  - A saída pode ser qualquer uma das n! possíveis permutações dos elementos de entrada
    - Idepende do algoritmo usado
    - A árvore de decisão deve conter n! folhas
- Observação: uma árvore binária possui, no máximo, 2^{h-1} folhas
- Então a árvore de decisão possui $n! \leq 2^{h-1}$, o que implica em $h \geq \lg(n!) + 1 = \Omega(n \lg n)$
- No pior caso, qualquer algoritmo é $\Omega(n \lg n)$

Exercícios

- Livro Szwarcfiter:
  - 7.1 a 7.13